

Build PostProcessing & Scripting Define Symbols

PostProcessing: iOS

During build postprocess:

- Adds frameworks:
 - `CoreMIDI.framework`
 - `CoreAudioKit.framework`
- Adjusts `Info.plist` :
 - adds `NSBluetoothAlwaysUsageDescription`

PostProcessing: Android

During build postprocess:

- Adjusts `AndroidManifest.xml` with permissions:
 - `android.permission.BLUETOOTH`
 - `android.permission.BLUETOOTH_ADMIN`
 - `android.permission.ACCESS_FINE_LOCATION`
 - `android.permission.BLUETOOTH_SCAN`
 - `android.permission.BLUETOOTH_CONNECT`
 - `android.permission.BLUETOOTH_ADVERTISE`
- Adds required features:
 - `android.hardware.bluetooth_le`
 - `android.hardware.usb.host`

Meta Quest (Oculus Quest): USB MIDI device detection

If you want USB MIDI on Meta Quest devices, you must enable the USB intent filter during postprocess.

In `PostProcessBuild.cs`, uncomment the line that adds the Oculus USB intent filter:

```
// androidManifest.AddUsbIntentFilterForOculusDevices();
```

Android: CompanionDeviceManager for BLE MIDI

You can use Android's Companion Device Pairing for BLE MIDI device connection.

To enable:

- Add scripting define symbol: `FEATURE_ANDROID_COMPANION_DEVICE`
- Unity path: Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols

Notes:

- Meta Quest devices can use this feature to find/connect Bluetooth MIDI devices.
- This feature may require requesting location permission depending on Android version/behavior.
- On Unity 6+ (2023.1+), when **Application Entry Point** includes **GameActivity**, the post-process sets the main activity to `jp.kshoji.unity.midi.BleMidiUnityGamePlayerActivity`. When only **Activity** is selected, it uses `jp.kshoji.unity.midi.BleMidiUnityPlayerActivity`. If both entry points are enabled, each Unity launcher activity is rewritten to the matching BLE MIDI activity.

Nearby Connections MIDI (Google Nearby)

Add dependency package

In Unity Package Manager:

- click `+`
- choose **Add package from git URL...**
- enter one of:
 - `git+https://github.com/kshoji/Nearby-Connections-for-Unity`
 - (SSH alternative) `ssh://git@github.com/kshoji/Nearby-Connections-for-Unity.git`

If already installed, update to latest.

Enable scripting define symbol

Add:

- `ENABLE_NEARBY_CONNECTIONS`

Android project setting

Set Target API level to 33 or higher:

- `Project Settings > Player > Identification > Target API Level`

Usage overview

Advertising:

- `MidiManager.Instance.StartNearbyAdvertising()`
- `MidiManager.Instance.StopNearbyAdvertising()`

Discovering:

- `MidiManager.Instance.StartNearbyDiscovering()`
- `MidiManager.Instance.StopNearbyDiscovering()`

Sending/receiving MIDI data is the same as normal MIDI once connected.

Maestro / MPTK integration (optional)

The plugin includes an optional integration layer for **Maestro / MidiPlayerTK (MPTK)**.

To enable it, add the scripting define symbol:

- `FEATURE_USE_MPTK`

Unity path:

- `Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols`

What it enables:

- An MPTK-backed **virtual MIDI output** device (route `MidiManager` sends into an MPTK synth).
- An adapter that can treat MPTK players as a **virtual MIDI input** source (injecting events into `MidiManager`).

Notes:

- Only enable this symbol when the MPTK asset is present in the project; otherwise compilation will fail due to missing MPTK types.
- See also: [Maestro / MPTK Integration](#)

Unity integration packages (optional)

Timeline and Visual Scripting integration require **both** installing the corresponding Unity package and adding a scripting define symbol.

Animator integration needs no extra setup (it is included in the core asmdef `jp.kshoji.midi`).

Unity symbol path:

- Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols

Timeline integration

1. Install **Timeline** via Package Manager:

- `com.unity.timeline`

2. Add the scripting define symbol:

- `FEATURE_USE_TIMELINE`

What it enables:

- Assembly Definition `jp.kshoji.midi.timeline` / `jp.kshoji.midi.timeline.editor`
- Timeline integration components such as `MidiPlaybackTrack` / `MidiRecordTrack`
- Sample scene `MidiTimelineIntegrationSampleScene`

Notes:

- Enabling only the symbol without installing the package causes `Unity.Timeline` reference errors.
- When the symbol is disabled, the Timeline integration code is excluded from compilation (the core plugin still builds).

Visual Scripting integration

1. Install **Visual Scripting** via Package Manager:

- `com.unity.visualscripting`

2. Add the scripting define symbol:

- `FEATURE_USE_VISUALSCRIPTING`

What it enables:

- Assembly Definition `jp.kshoji.midi.visualscripting`
- `MidiVisualScriptingBridge` and the MIDI custom node set
- Sample scene `MidiVisualScriptingIntegrationSampleScene`

Notes:

- Enabling only the symbol without installing the package causes `Unity.VisualScripting` reference errors.
- When the symbol is disabled, the Visual Scripting integration code is excluded from compilation.

Scriptable Audio Pipeline integration (optional)

The plugin includes an optional integration layer for Unity 6.3+ [Scriptable Audio Pipeline](#).

Requirements:

- **Unity 6000.3 LTS** or newer
- Supported platform (not WebGL)

To enable, add the scripting define symbol:

- `FEATURE_SCRIPTABLE_AUDIO`

Unity path:

- `Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols`

Optional packages (included in this repo's `Packages/manifest.json` when using the full project):

- `com.unity.burst`
- `com.unity.collections`

What it enables:

- Assembly Definition `jp.kshoji.midi.scriptableaudio`
- `ScriptableAudioBootstrap`, `MidiMetronomeGenerator`, `MidiDspClockBridge`, and Pipe/DSP clock infrastructure
- DSP-scheduled SMF playback: `MidiDspSequenceScheduler`, `MidiSequenceSynthGenerator`, `MidiDspSequenceBootstrap`
- DSP-scheduled UMP playback: `MidiDspUmpSequenceScheduler`, `UmpSequenceSynthGenerator`, `MidiDspUmpSequenceBootstrap`
- Sample scenes `ScriptableAudioMetronomeSampleScene`, `ScriptableAudioSequenceSampleScene`, and `ScriptableAudioUmpSequenceSampleScene`

Quick start after enabling:

1. **Metronome:** Add **Scriptable Audio Bootstrap** to a GameObject, or open `ScriptableAudioMetronomeSampleScene.unity`.
2. Enter Play Mode — you should hear 120 BPM metronome clicks.

SMF sequence (DSP-synced playback):

1. Add **Midi Dsp Sequence Bootstrap** (`MidiDspSequenceBootstrap`) and assign a `MidiSequenceAsset`, or open `ScriptableAudioSequenceSampleScene.unity`.

2. Enter Play Mode — notes are scheduled on the Unity DSP clock via the built-in reference synth.

UMP sequence (DSP-synced playback):

1. Add **Midi Dsp Ump Sequence Bootstrap** (`MidiDspUmpSequenceBootstrap`) and assign a `UmpSequence` , or open `ScriptableAudioUmpSequenceSampleScene.unity` .
2. Enter Play Mode — UMP clip notes are scheduled on the Unity DSP clock via the built-in reference synth.

Notes:

- When the symbol is undefined, or on Unity versions before 6.3, or on WebGL, the integration assembly is not compiled; core `jp.kshoji.midi` still builds.
- **SmfPlayer** (frame-driven, MIDI device output) and **MidiDspSequenceScheduler** (DSP-sample-accurate built-in audio) are separate; use one per use case.
- **UmpSequencer** (wall-clock, dedicated thread) and **MidiDspUmpSequenceScheduler** (DSP-synced built-in audio) are separate; use the former for clip editing / trial playback and the latter for in-game BGM, loops, and seek.
- Run **Window > MIDI > Validate Scriptable Audio Setup** to diagnose metronome, SMF, or UMP bootstrap wiring in the Editor.
- See also: [Integrations \(Japanese\)](#) and [Scriptable Audio Integration](#).

Animator integration

No extra package or scripting define symbol is required.

See [Unity Ecosystem Integrations](#) for details.

Network optional kit

Network MIDI synchronization requires adding a scripting define symbol.

Unity symbol path:

- Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols

Network MIDI synchronization

Scripting define symbol:

- FEATURE_MIDI_NETWORK

What it enables:

- Assembly Definition jp.kshoji.midi.net
- MidiNetworkHub / MidiNetworkClient / MidiPlaybackSync
- Sample MidiNetworkJamSampleScene

Notes:

- The UDP ports (default 55000–55002) must be allowed through the firewall.
- When the symbol is disabled, the relevant code is excluded from compilation.

Optional: Mirror / Netcode / WNet2 bridges

In addition to FEATURE_MIDI_NETWORK, optional bridges compile only when their symbols (and package gates) are present. Framework packages are **not** vendored.

Symbol	Description
FEATURE_MIRROR	Mirror (MidiMirrorBridge). Also needs MIRROR (from Mirror)
FEATURE_NETCODE	Netcode for GameObjects (MidiNetcodeBridge). MIDI_HAS_NETCODE via UPM versionDefines
FEATURE_WSNET2	WSNet2 (MidiWsn2Bridge). MIDI_HAS_WSNET2 when WNet2.Runtime.asmdef is present (Editor sync)

CI (BatchCompileBuilder) registers FEATURE_MIRROR / FEATURE_NETCODE / FEATURE_WSNET2 with companion FEATURE_MIDI_NETWORK. Without the package gate, the bridge asm is excluded so core net still builds.

Flow: install framework → enable symbols → wire components → open sample scene. Details: [Genre Kits](#), [Integrations](#), [Networking Integration](#).

The UDP-based `MidiNetworkHub` / `MidiNetworkClient` can be used without the above.

Cross-cutting optional kit

The Input System bridge requires adding a scripting define symbol.

MidiClockSync and chord/scale detection require no symbol (core `jp.kshoji.midi`).

Unity symbol path:

- Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols

Input System bridge

1. Add `com.unity.inputsystem` via Package Manager (not needed if already bundled in `Packages/manifest.json`).
2. Add the scripting define symbol:
 - `FEATURE_INPUT_SYSTEM`

What it enables:

- Assembly Definition `jp.kshoji.midi.inputsystem`
- `MidiInputSystemBridge` / `InputSystemToMidiBridge` / `MidiSyntheticDevice`
- Sample `InputSystemBridgeSampleScene`

Notes:

- When the symbol is disabled, the Input System integration code is excluded from compilation (the core plugin still builds).

MidiClockSync / chord & scale detection

No additional symbol is required; it is included in the core asmdef (`jp.kshoji.midi`).

See [Genre Kits — Cross-cutting](#) for details.

Chunity (Chuck) integration (optional)

This plugin **does not bundle the Chunity runtime**. The integration code is compiled only when the user installs it separately and enables the symbol.

1. Install [Chunity](#) into your project.

2. Copy

`Assets/MIDI/Scripts/Integrations/Chunity/Optional/Chunity.Runtime.asmdef.example` to the Chunity Scripts root as `Chunity.Runtime.asmdef` (e.g., `Assets/Chunity/Scripts/Chunity.Runtime.asmdef`). See the [Chunity Optional patch guide](#) for details.

3. Add the scripting define symbol:

- `FEATURE_CHUNITY`

What it enables:

- Assembly Definition `jp.kshoji.midi.chunity`
- `MidiChuckBridge` / `MidiChuckPatchHost` / `MidiChuckMapping` / Phase 2–3 components
- Samples `ChunityBridgeSampleScene` / `ChunityWorkflowsSampleScene` / `ChunityPresetsSampleScene`
- (Optional) `FEATURE_USE_TIMELINE` → `jp.kshoji.midi.chunity.timeline`
- (Optional) `FEATURE_USE_VISUALSCRIPTING` → `jp.kshoji.midi.chunity.visualscripting`

Notes:

- If only the symbol is enabled while Chunity is not installed (or the `Chunity.Runtime` asmdef is not placed), type resolution errors occur (same as MPTK).
- When the symbol is disabled, the integration code is excluded from compilation (the core plugin still builds).
- User-facing description: [Unity Ecosystem Integrations — Chunity](#)
- Setup details: [Chunity Integration](#)

Chunity Scriptable Audio Generator (optional)

Optional Chuck audio path using Unity 6.3+ Generators instead of `OnAudioFilterRead`. Control path (`MidiChuckBridge`, etc.) is unchanged.

Requirements:

- Baseline Chunity integration (`FEATURE_CHUNITY` + `Chunity.Runtime.asmdef`)
- Minimal Chunity patch: `useBuiltInAudioFilter` on Main/Sub (see [Chunity Optional patch guide](#))

- Unity package `com.unity.collections` (asmdef references `Unity.Collections` for `NativeArray` / `FixedString128Bytes` ; install via Package Manager if missing)
- Unity 6000.3+ (Generator path is not available on WebGL)

Add scripting define:

- `FEATURE_CHUNITY_SCRIPTABLE_AUDIO`

What it enables:

- Assembly Definition `jp.kshoji.midi.chunity.scriptableaudio`
- `ChuckMainGeneratorDriver` / `ChuckSubGeneratorDriver`
- Samples `ChunityGeneratorWorkflowSampleScene`

Notes:

- Distinct from `FEATURE_SCRIPTABLE_AUDIO` (metronome / sequence synths).
- Details: [Chunity Integration](#) and [Japanese build-postprocessing](#).